

UNIT III

Decision Making Statement

Introduction

The statements in a program are executed line by line similarly in the way we read a text book. The method to read a textbook is straight forward, which starts from first line to the end of line and from top to bottom.

Similarly, in the program the statements are read and executed sequentially line-by-line unto the end of program. In some time, we may select one of the options from the available options by testing some condition. Depending on the problem, the flow of the program is fixed.

C Language supports decision making to control the flow and execution of statements in any order. Such statements are decision or control statements. Decision-making is one of the important aspects of the computer. For making decisions computer needs a condition i.e logical expression and decision-making statement. For building conditions, we need the operators.

Expressions in decision-making statements normally consist of arithmetic, relational, logical or conditional operators.

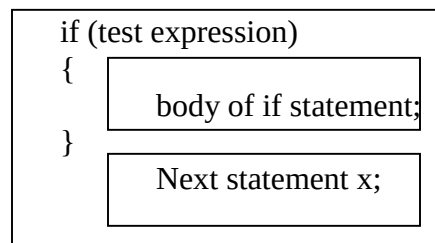
Decision making structures

- The if statement
- The if- else statement
- Nested if – else statement

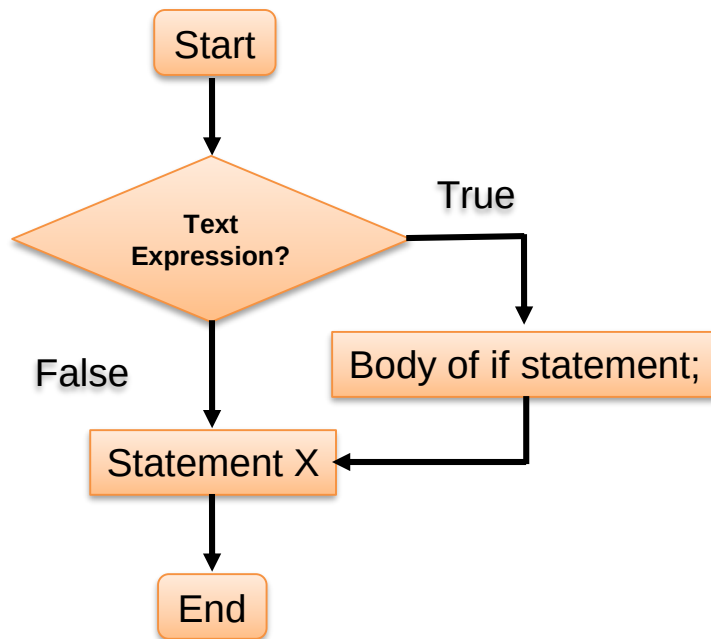
The if statement

If it is an keyword, The if statement is a powerful decision making statement used to control the flow of execution of statements. First condition is checked. If condition is true than it execute the true statement block control transfer to statement x. If the condition is false then without executing the true block the control is goes directly to the next statement x. The if statement always used in conjunction with expressions. These expressions evaluate true of false.

The general format of a simple if statement is



The body of if may have one statement or group of statement where each statement is separated by semicolon. If the test expression is true, the true statement block will be executed otherwise computer will skip the block of statement and the execution will jump to the next statement immediately after the end of if statement.



Simple If statement flow chart

```
If ( a>b)
{
    Printf("%d\n",a);
}
```

```
Void main()
{
    Int a,b;
    Printf("\nEnter the Number a & b");
    Scanf("%d%d",&a,&b);
    if(a>b)
        Printf("A is grater number");
    If(b>a)
        Printf("B is grater number");
    Getch();
}
```

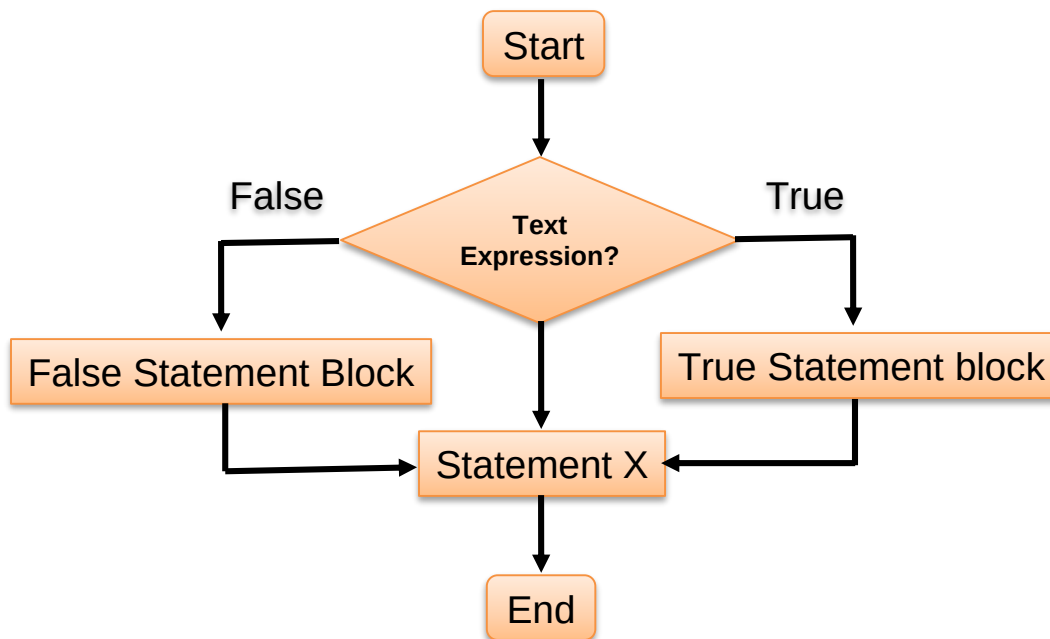
- The test expression (condition) should not end with semicolon(;).
- The test expression without expression arguments is not acceptable.
- The test expression must be always enclosed within a pair of brackets().

If – else statement

The if – else statement is an extension of the simple if statement. In this statement first check the condition if condition is true then body of if (true) statement is executed otherwise body of else (false) statement is executed. In this statement either body of if or body of else is executed and then only the control moves to the next statement.

Syntax:-

```
if (condition)
{
    Body of true statement block;
}
else
{
    Body of false statement block;
}
```



Flow Chart of if – else Statement

Ex:-

```
if (a>b)
{
    Printf("\n A is grater then B");
}
else
{
```

```
        Printf("\nA is less then B");  
    }
```

This statement provides the alternates in both the situations, if condition is true the body of if is executed if condition is false the body of else is executed.

Here computer can not proceed without executing either body of if or body of else.

//Write a program to find grater number within two numbers

```
void main()  
{ int a,b;  
    printf("\nEnter the Number a & b");  
    scanf("%d%d",&a,&b);  
    if(a>b)  
    {  
        printf("A is grater number");  
    }  
    else  
    {  
        printf("B is grater number");  
    }  
    getch();  
}
```

// Write a program to find given number is even or odd

```
void main()  
{ int num;  
    printf("\nEnter the any number");  
    scanf("%d",&num);  
    if(num%2==0)  
    {  
        printf("The given number is even number");  
    }  
    else  
    {  
        printf("The given number is odd number");  
    }  
    getch();  
}
```

//Write a program to find given year is leap year or not

```
void main()  
{ int year;  
    printf("\nEnter the year");  
    scanf("%d",&year);  
    if(year%4==0)  
    {  
        printf("The given year is leap year");  
    }  
    else  
    {  
        printf("The given year is not leap year");  
    }  
}
```

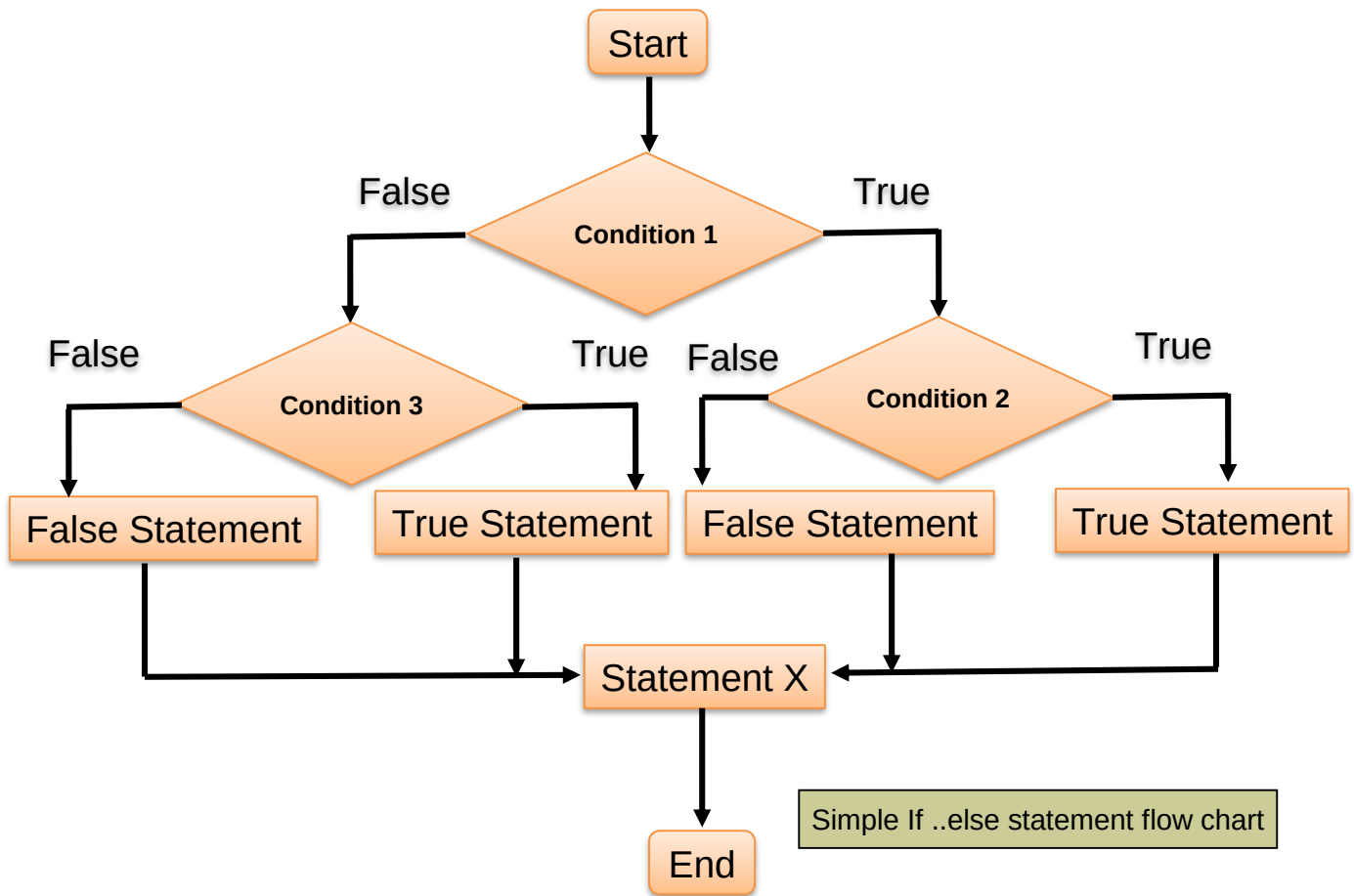
```
    }getch();  
}
```

Nested if –else statement

It is perfectly all right if we write an entire if-else construct within either the body of the if statement or the body of an else statement. This is called as nesting of if. In this kind of statement no of logical condition are checked. For executing various statement. Nested if-else can be chained with one another If the condition is check if condition is false then control passes to else statement block where condition is again checked with if statement. This process continue become condition is true if condition is true then execute the true statement block otherwise last else statement block will be execute.

Syntax:-

```
if(Condition 1)  
{  
    if(Condition 2)  
    {  
        Body of True Statement Block  
    }  
    else  
    {  
        Body of False Statement Block  
    }  
}  
else  
{  
    if(Condition 3)  
    {  
        Body of True Statement Block  
    }  
    else  
    {  
        Body of False Statement Block  
    }  
}
```



Flow Chart of Nested if – else Statement

```
Ex-
if(a>b)
{
    if(a>c
    {
        printf("a is grater");
    }
    else
    {
        printf(c is grater");
    }
}
```

```
else
{
    if(b>c)
    {
        printf("B is grater");
    }
    else{
        printf("c is grater");
    }
}
```

In that first check the condition. If the given condition1 is true then again check the condition 2 if the given condition 2 is true then execute the true statement block and control

transfer to statement x. if the given condition 2 is false then without executing the true statement block execute the false statement block of condition 2 and control transfer to statement x. if given condition 1 is false then control transfer to else statement again check the condition 3 if the condition 3 is true then execute the true statement and control transfer to statement x. if the given condition 3 is false then execute the false statement block and control transfer to statement x.

When this structure gets executed and as soon as any one of the statement gets executed, immediately the chain breaks & computer goes next statement. There are many possible forms of nesting. It always depends on our problem.

Write a program that display the class obtained by student if percentage of the student is input through the keyboard.

```
void main()
{
    float per;
    printf("\nEnter the percentage");
    scanf("%f",&per);
    If(per<0 && per>100)
    {
        printf("Invalid input");
        exit();
    }

    if(per<40)
        printf("Fail");
    else
    {
        if(per<50)
            printf("Pass");
        else
        {
            if(per<60)
                printf("Second class class");
            else
                printf("First class");
        }
    }
}
```

Write a program find out entered key is upper case or lower case.

```
#include<stdio.h>
#include<conio.h>
void main()
{
char x;
clrscr();
printf("\nEnter any char");
scanf("%c",&x);
if((x>=65&&x<=90)|| (x>=97&&x<=122))
{
if(x>=65&&x<=90)
printf("it is an upper case alphabet");
else
printf("It is lower case alphabet");
}
else
printf("It is not alphabet");
getch();
}
```

Positive number and negative number

```
#include<stdio.h>
#include<conio.h>
void main ()
{
int num;
printf("\nEnter any number");
scanf("%d",&num);
if(num>0)

{
printf("given number is positive");
}
else
{
if(num<0)
{
printf("given num is negative");
}
else
{
printf("Zero");
}
}
getch();}
```

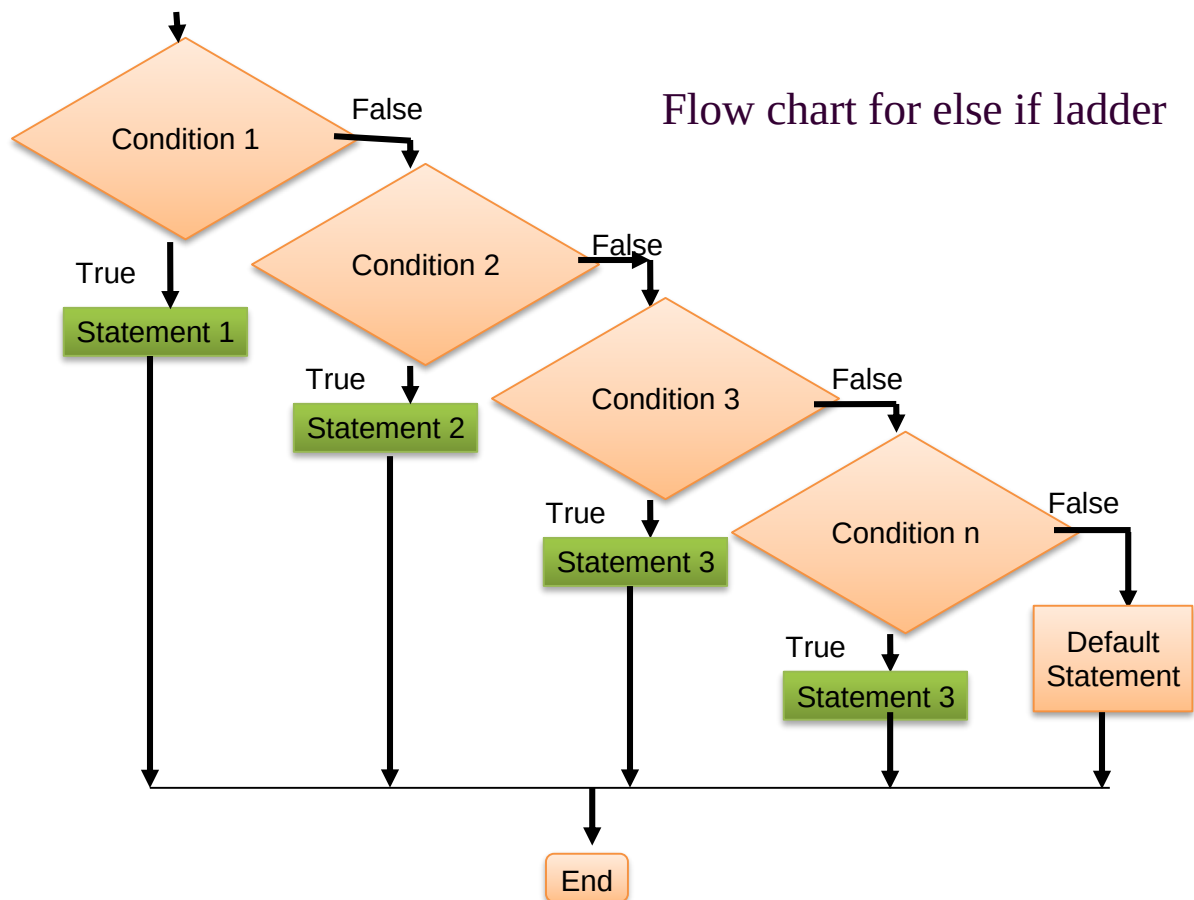

else—if ladder

There is a negative side to the program. Even if the first condition turns out to be true, still all other condition are checked. This will increase the time of execution of the program. This can be avoided using the else if clause.

When multiple decision are taken then else if is used when we used else if ladder, can else is associated with if.

Syntax

```
If(Condition 1)
    statement 1;
else if(condition 2)
    statement 2;
else if(condition 3)
    statement 3;
else if(condition n)
    statement n;
else
    default statement;
statement x;
```



To find the grade

```
Void main()
{
    Float per;
    Printf("Enter the value");
    Scanf("%f",&per);
    If(per>80)
    {
        Printf("grade is merit");
    }
    Else if(per>70)
    {
        Printf("Distinction");
    }
    Else if(per>60)
    {
        Printf("Grade A");
    }

    else if(per>50)
    {
        printf("Grade is B");
    }
    else if(per>=35)
    {
        printf("Grade is C");
    }
    else
    {
        printf("Failed");
    }
    printf("try try but don't cry, one day you will success");
    getch();
}
```

Write a program to Larger number within five number.

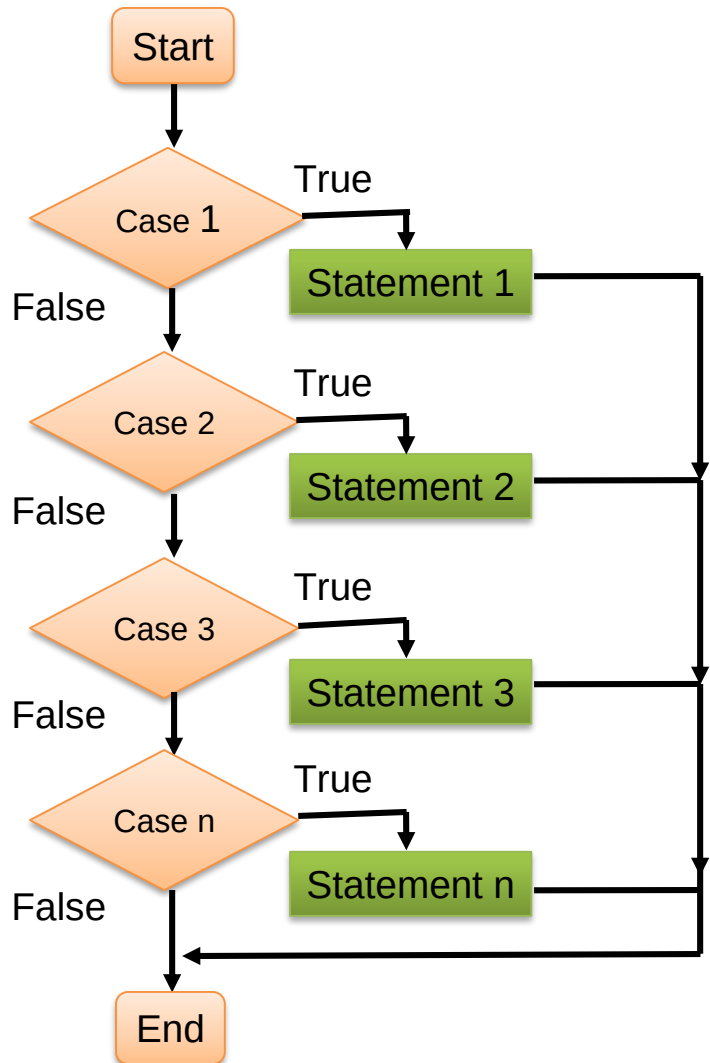
```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n1,n2,n3,n4,n5,larg,small;
    clrscr();
    printf("\nEnter the number");
    scanf("%d%d%d%d%d",&n1,&n2,&n3,&n4,&n5);
    larg=n1;
    if(n1>larg)
    larg=n1;
    if(n2>larg)
    larg=n2;
    if(n3>larg)
    larg=n3;
    if(n4>larg)
    larg=n4;
    if(n5>larg)
    larg=n5;
    printf("\nthe largest number within five number is %d",larg);
    small=n1;
    if(n1<small)
    small=n1;
    if(n2<small)
    small=n2;
    if(n3<small)
    small=n3;
    if(n4<small)
    small=n4;
    if(n5<small)
    small=n5;
    printf("\nthe smallest number within five number is %d",small);
    getch();}
```

Switch statement

By using switch statement, which have multiple collection but (alternative increases) complexity increase and program become easy to read and debug. Switch statement is multi-case statement. The keyword switch statement value of condition match with label
Syntax:

```
switch(expression)
{
    case value 1:
        statement 1;
        break;
    case value 2:
        statement 2;
        break;
    Default:
        Default block;
}
```

Switch flow chart



If condition is doesn't match with label of case (1) than it checks condition value is match with label of case (2). If that value is match than execute statement block of case (2) and control transfer to statement x and so on---

We have to remember that, case labels are single character constants.

Explanation of switch statement

Expression: it is an integer expression or character.

Value1, value 2 : this are constants or constants expression known as case label.

Block 1, block 2: this are statement which may content one or more statement.

Case label must be end with colon(:).

default: the default is an optional case which it will execute expression value, doesn't match with any case value.

- In a switch there can be either variable or expression
- If it is a variable it must be either integer or character
- If it is an expression it must be an arithmetic expression.
- There can be any number of cases.
- Every case should have an unique value.
- These cases can be written in any sequence.
- In a case there can be any number of statement.
- It is possible to have the nested switch.
- After every case break statement is compulsory.
- Default is a case which gets selected when none of the case value matches with the result of expression.
- Default case is optional.

Write a program that reads a number between 1 to 7 and display the day name

```
main()
{
    Int day;
    Printf("Enter a number between 1 to 7 \n");
    Scanf("%d",&day);
    Switch(day)
    {
        Case 1:printf("Monday\n");
        Break;
        Case 2:printf("Tuesday\n");
        Break;

        Case 3:printf("Wednesday\n");
        Break;

        Case 4:printf("Thursday\n");
```

```
Break;

Case 5:printf("Friday\n");
Break;
Case 6:printf("Saturday\n");
Break;
Case 7:printf("Sunday\n");
Break;
Default:printf("Monday\n");
}
```

Write a program add,sub,multi & division of two number

```
void main()
{
int a,b,n,p;
float c;
clrscr();
xyz:printf("\nEnter two number");
scanf("%d%d",&a,&b);
printf("\n 1 addition of two number");
printf("\n 2 subtraction of two number");
printf("\n 3 multiplication of two number");
printf("\n 4 division of two number");
printf("\nEnter your choice");
scanf("%d",&n);

switch(n)
{
case 1:c=a+b;
printf("\n addition of two number is%f",c);
break;
case 2:c=a-b;
printf("\n subtraction of two number is%f",c);
break;

case 3:c=a*b;
printf("\n multiplication of two number is%f",c);
break;
case 4:c=a/b;
printf("\n division of two number is%f",c);
break;
default:
printf("\n entered choice incorrect");
break;
}
```

Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

```
    }
    printf("\ndo you want to continue");
    printf("\nenter 6 yes 7: no");
    scanf("%d",&p);

    switch(p)
    {
    case 6:goto xyz;
    case 7:exit();
    }
    getch();
}

#include<stdio.h>
#include<conio.h>
#include<process.h>
void main()
{
    int a,b,c,n,d;
    clrscr();
    xyz:printf("\nEnter the number a");
    scanf("%d",&a);
    printf("\nEnter the number b");
    scanf("%d",&b);
    printf("\n\n 1 Addition");
    printf("\n\n 2 Subtraction");
    printf("\n\n 3 Multiplication");
    printf("\n\n 4 Divission");
    printf("\n\n Enter the choice");
    scanf("%d",&n);
    switch(n)
    {
    case 1:c=a+b;
    printf("\n\nAddition of two number is%d",c);
    break;
    case 2:
    c=a-b;
    printf("\n\nsubtraction of two number is%d",c);
    break;
    case 3:
    c=a*b;
    printf("\n\nmulti of two number is%d",c);
    break;
    default:
    c=a/b;
```

Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

```
printf("\n\ndivision of two number is%d",c);  
}  
printf("\n\nEnter 6 to Yes and enter 7 No");  
scanf("%d",&d);  
switch(d)  
{  
case 6:goto xyz;  
case 7:exit(0);  
break;  
}  
getch();  
}
```

The output of the following code is _____

```
#include <stdio.h>  
int main()  
{  
int c=1;  
switch( c ){  
case 1:  
printf("1");  
default:  
printf("0");  
}  
return 0;  
}
```

Answer: 10

Looping Statement

In the program so many times we come across the situation where we want to execute some part of the program again and again. For some particular number of time. In such situations we make use of loop statements. These are the statements which can execute a block of statements for some particular number of time.

The loops are used for repetitive execution of statement in block of loop as long as the condition is true, all statements in a block of loop are executed repeatedly. When looping condition become false. The control moves to the next statement immediately after the end of loop block. Here the condition is a valid logical statement. The logical statement determines the condition under which the loop should continue repeating.

So when we want to repeat some part or procedure for some particular number of times then we use loops.

In this language there are three loops

- while loop
- do-while loop
- for loop
- break statement, continue statement, goto statement

Entry control loop

In this loop, first condition is checked, if the condition is true, than it executes the body of loop. Again the condition is checked, if the condition is true, it execute body of loop . So on , this processes continues until the given condition become false. Once condition become false, than it transfer control to statement x without executing the body of loop

Ex -While loop,For loop

Exit control loop

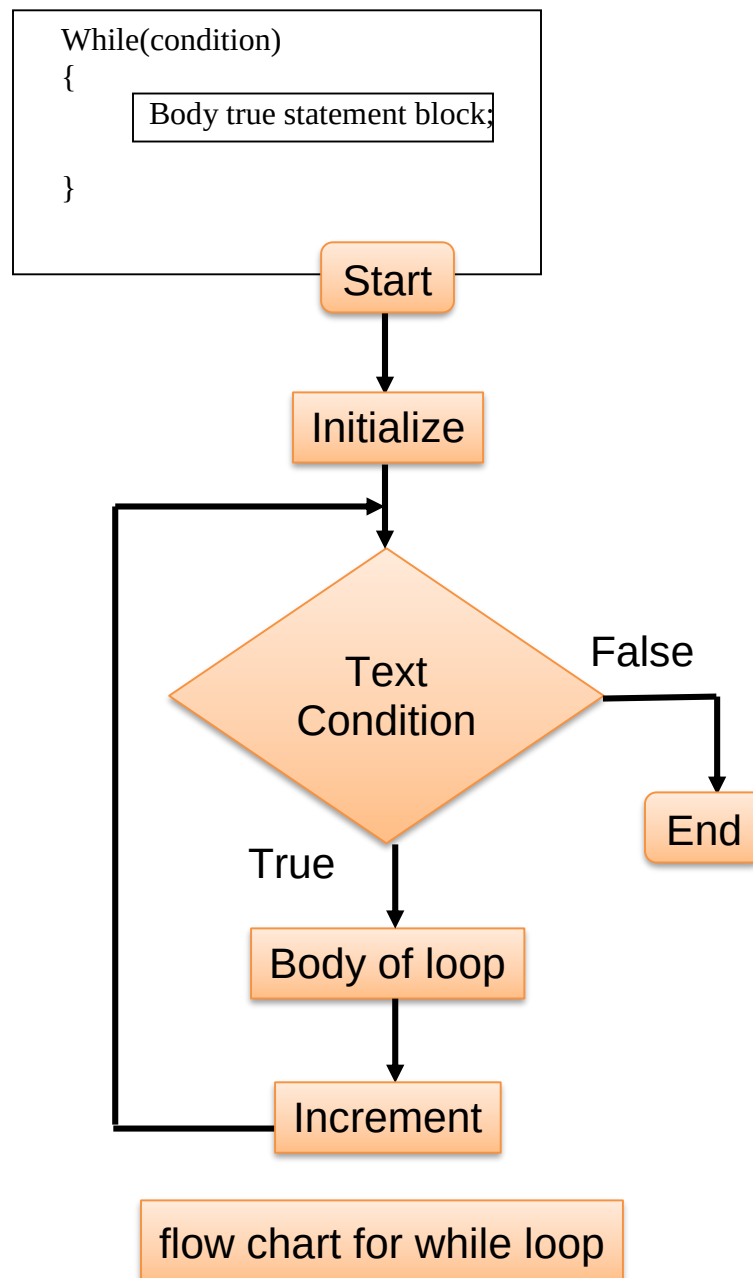
In this loop, first body of loop is executed, then the condition is checked. If the condition is true, than it execute body of loop. Again condition is checked, if it is true the body of execute. So on this process continue until the given condition become false. Once condition become false. It control transfer to statement x without executing the body of loop.

In exit control loop, the condition may true or false, at least once the body of loop is executed

Ex-do-while

While loop

the while loop is one of the entry control loop. In this loop the condition is given at the top. First the check condition and if the condition is true then execute body of loop. The body of while may have one statement or more than one statement, if there is a single statement no need of brackets ' {}'. If there are more one statement then brackets are compulsory. Again the check condition and if it is true then execute body of loop this process still continues of condition become false if condition become false does not execute the body of loop and control goes to statement x. This loop should be used when condition is more important. Syntax:



Ex:-

```
void main()
{
    int i=1;
    clrscr();
    while(i<=100)
    {
        printf("%d\n",i);
        i=i+1;
    }getch();
}

void main()
{
    int i=2;
    clrscr();
    while(i<=100)
    {
        printf("%d\n",i);
        i=i+2;
    }

    getch();
}
```

Do while

There is a minor difference between the working of while and do-while loops this difference is the place where the condition is tested. The while tests the condition before executing any of the statements within the while loop. The do-while tests the condition after having executed the statements within the loop.

Syntax:-

```
Do
{
    Body of the loop
}
While(test-condition);
```

Since the test condition is execute at the bottom of the loop, the do –while construct provides an exit controlled loop and therefore the body of the loop is always executed at least once.

```
#include<stdio.h>
#include<conio.h>
#include<math.h>
```

```
void main()
{
```

Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

```
int n,k,m;
clrscr();
scanf("%d",&k);
n=1;
do
{
    m=pow(n,2);
    printf("%d\n",m);
    n++;
} while(m<=(pow(k,2)));
getch();
}

//display 5,10,15----
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,k;
    clrscr();
    scanf("%d",&k);
    n=5;
    while(n<=k)
    {
        printf("%d\n",n);
        n=n+5;
    }
    getch();
}

//display 7,14,21----
#include<stdio.h>
#include<conio.h>
void main()
{
    int n=7;
    clrscr();
    while(n<=70)
    {
        printf("%d\n",n);
        n=n+7;
    }
    getch();
}

//display 10,20,30----
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,k;
    clrscr();
    scanf("%d",&k);
    n=10;
    while(n<=k)
    {
        printf("%d\n",n);
        n=n+10;
    }
    getch();
}

//display 1,2,4,7,11----
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,k,d;
    clrscr();
    scanf("%d",&k);
    n=1;
    d=1;
    while(n<=k)
    {
        printf("%d\n",n);
        n=n+d;
        d=d+1;
    }
    getch();
}

//display 1,2,4,8----
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,k;
    clrscr();
    scanf("%d",&k);
    n=1;
    while(n<=k)
    {
        printf("%d\n",n);
        n=n*2;
    }
    getch();
}
```

Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

//sum of 1 to 10 number

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i=1,sum=0;
    clrscr();
    while(i<=10)
    {
        printf("%d\n",i);
        sum=sum+i;
        i++;
    }
    Printf("sum of I to 10 number%d",sum);
    getch();
}
```

//multiplication table

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int I,j;
    clrscr();
    while(i<=10)
    {
        j=1;
        while(j<=10)
        {
            printf("%d\t",i);
            j++;
        }
        printf("\n");
        i++;
    }
    Printf("sum of I to 10 number%d",sum);
    getch();
}
```

Difference between while and do-while

While	Do—while
Condition is at the top	Condition is at the bottom.
No necessity of brackets if there is single statement in body.	Brackets are compulsory even if there is a single statement.
There is no semicolon at the end of while.	The semicolon is compulsory at the end of do-while.
Computer executes the body if and only if condition is true.	Computer executes the body at least once even if condition is false
This should be used when condition is more important.	This should be used when the process is important.
This loop is also referred as entry controlled loop.	This loop is also referred as exit controlled loop.
<pre>while(n<=10) { printf(“%d\n”,n); n++; }</pre>	<pre>do { printf(“%d\n”,n); n++; }while(n<=100);</pre>

```
void main()
{
    int i=1;
    clrscr();
    do
    {
        printf(“%d\n”,i);
        i=i++;
    } while(i<=100);

    getch();
}
```

//display 5,10,15----

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,k;
    clrscr();
    scanf("%d",&k);
    n=5;
    do
    {
        printf("%d\n",n);
        n=n+5;
    } while(n<=k);
    getch();
}
```

//display 7,14,21----

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n=7;
    clrscr();
    do
    {
        printf("%d\n",n);
        n=n+7;
    } while(n<=70);
    getch();
}
```

//display 10,20,30----

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,k;
    clrscr();
    scanf("%d",&k);
    n=10;
    do
    {
        printf("%d\n",n);
```


Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

```
        n=n+10;
    } while(n<=k);
    getch();
}
```

//display

1,2,4,7,11----

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,k,d;
    clrscr();
    scanf("%d",&k);
    n=1;
    d=1;
    do
    {
        printf("%d\n",n);
        n=n+d;
        d=d+1;
    } while(n<=k);
    getch();
}
```

//display 1,2,4,8----

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,k;
    clrscr();
    scanf("%d",&k);
    n=1;
    do
    {
        printf("%d\n",n);
        n=n*2;
    } while(n<=k);
    getch();
}
```

//display 1,2,4,8----

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int n,km;
    clrscr();
    scanf("%d",&k);
    n=1;
    do
    {
        m=pow(n,2);
        printf("%d\n",m);
        n++;
    } while(m<=(pow(k,2)));
    getch();
}
```

//multiplaction table

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i=1,j;
    clrscr();
    while(i<=5)
    {
        j=1;
        while(j<=5)
        {
            printf("%d\t",i*j);
            j++;
        }
        printf("\n\n");
        i++;
    }
    getch();
}
```

Display output as

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i=1,j;
    clrscr();
    while(i<=5)
    {
        j=1;
        while(j<=i)
        {
            printf("*\t");
            j++;
        }
        printf("\n\n");
        i++;
    }
    getch();
}
```

```
*
*
* * *
* * * *
```

For loop

This statement is used when calculations are to be carried out between initial and final value with step. The for loop condition consist of three actions namely initialization, testing and incrementing/ decrementing. Each action should be separated by semicolon(;) also this loop statement is used when number of execution time is known in advance.

It is an entry control loop having syntax as follow.

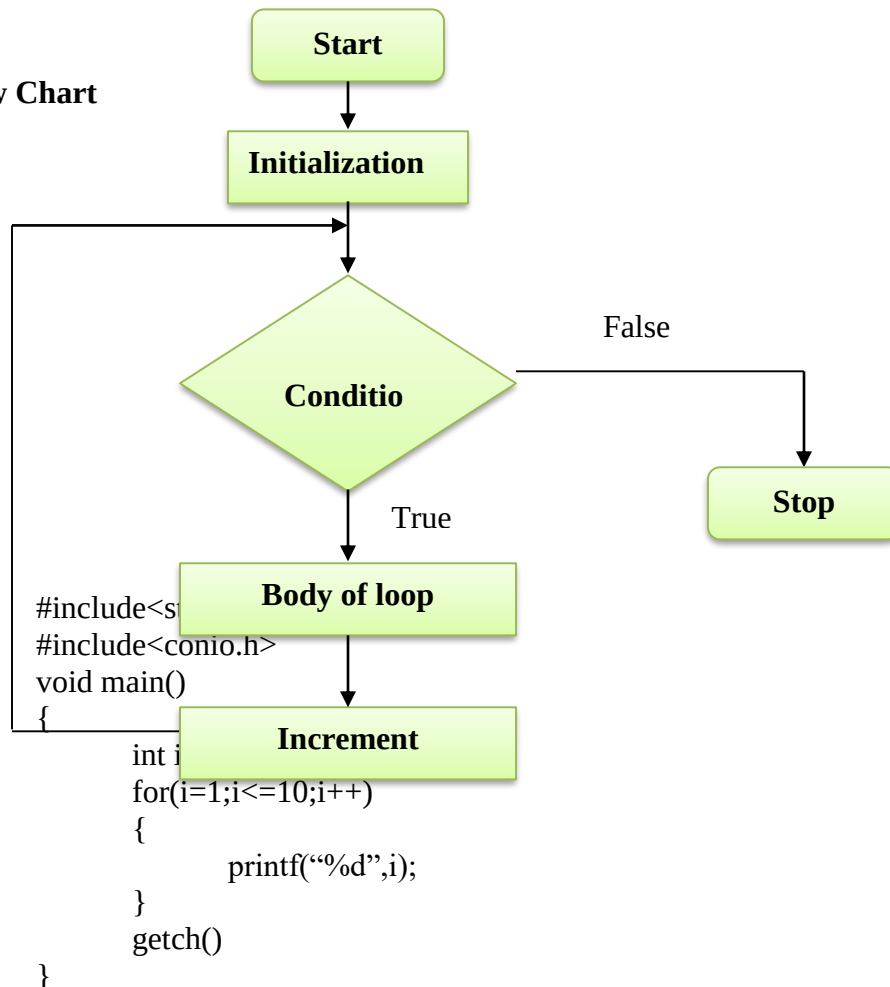
Syntax-

<pre>for(initial value;condition;increment/decrement) { body of for loop; }</pre>

In this loop the initial value, condition, increment/decrement is specified in the same line. Here computer starts with the initial value, checks the condition. If the condition is true computer executes the body and automatically goes up, it takes the increment or decrement automatically and it continues. Remember, if the body of for continue a single statement, no need of brackets. But if there are more than one statement in the body then they must be enclosed between brackets.

Here every thing is specified in the same line and hence we can tell the number of repetitions it can take. When we know the number of repetitions in advance than we should go for the for loop.

Flow Chart



Break Statement

The break statement is used for two different purpose it can be used to terminated a case in the switch statement that is to terminate the given loop immediately and it by pass the normal loop condition test.

we often come across situations where we want to jump out of a loop instantly, without waiting to get back to the conditional test, the keyword `break` allows us to do this. When `break` is encountered inside any loop, control automatically passes to the first statement after the loop. A `break` is usually associated.

Ex:-

```
Switch()
{
    Case 1:
    Break;
    Case 2:
    Break;
    Default:
}
ex:- void main()
{
    int I;
    for(i=1;i<=10;i++)
    {
        if(i==5)
        {
            break;
        }
        printf("%d",i);
    }
    getch();
}
```

Continue Statement:-

This statement is used to when execution of further statement. This stops execution of next statement & transfer control to the start of loop for further execution of loop. This statement should be used only within the loop.

Ex:- program to determine continue statement.

```
Void main()
{
    int i;
    for(i=1;i<=10;i++)
    {
        if(i==5)
        {
```

```
        continue;
    }
    printf("\n%d",i);
    getch();
}
```

in the above ex I is initialized to 1 then it will check (i==5) if it is true it will execute printf() statement. After execution I is incremented by 1 & again executes printf() statement. This process continues till the condition i<=10 becomes true.

Goto Statement:

This statement transfers the control from one statement to other statement in the program which may not be in the sequence. The general form of the goto statement is

```
goto label;
-----
label:
```

towards jump

```
label: statement
-----
goto label;
```

backward jump

where goto requires a level in order to identify the place where the branch is to be made label. It is an valid variable name & must be followed by a (:) the program either before or after the goto label statement as shown below.

Forward jump:

If the label is placed after goto label statement will be skipped such a jump is known as forward jump.

Backward jump:

Royal Education Society's
COLLEGE OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY, LATUR

If the label is before the goto statement a loop will form & some statement will be executed repeatedly such a jump is known as backward jump.

```
#include<stdio.h>
#include<conio.h>
#include<process.h>
void main ()
{
    int a,b;
    clrscr();
    xyz:printf("enter the number");
    scanf("%d",&a);
    if(a%2==0)
    {
        printf("given num is even ");
    }
    else
    {
        printf("\nthe given num is odd");
    }
    printf("\nif you want to continue");
    printf("\n Enter 6 to yes 7 to no");
    scanf("%d",&b);
    if(b==6) goto xyz;
    if(b==7) exit(0);
    getch();
}
```